

Monotonic High-Order Diffusion on NVIDIA Grace

STEFANIE BOERSIG, BOAZ KO, AND BEN BULLINGER

Department of Environmental Systems Sciences, ETH Zurich, Switzerland

(Report for High Performance Computing for Weather and Climate, 701-1270-00L; 12 August 2026)

ABSTRACT

We study the cost of enforcing monotonicity in high-order diffusion filters used in atmospheric models. These filters are known to produce unphysical overshoots when applied to sharp gradients. We implement the simple flux limiter proposed by Xue (2000) in the MPI stencil2d code for orders 2, 4, 6, and 8, with a normalization that removes the shortest resolvable ($2\Delta x$) mode in one step for all orders. We measure performance on a 72-core NVIDIA Grace CPU. Notably, the limiter’s absolute overhead is roughly constant with order, so relative cost drops from 66% at $n=2$ to 20% at $n=8$: the limiter is least expensive when the filter is most selective. We also explore data-locality optimizations: array demotion gives up to $3.3\times$ speedup, but only when the working set exceeds cache. A branch-free implementation is critical because conditional branches prevent vectorization. A GT4Py version matches the Fortran bit-for-bit and sees no limiter overhead on an H100 GPU. The Zalesak flux-correction method yields nearly indistinguishable solutions but costs $1.9\text{--}2.2\times$ more, providing the timing evidence that Xue’s paper asserted but didn’t measure.

1. Introduction

Atmospheric models add an artificial diffusion term to their prognostic equations,

$$\partial_t \phi = \dots + (-1)^{n/2+1} \alpha_n \nabla^n \phi, \quad n = 2, 4, 6, \dots, \quad (1)$$

to remove small-scale noise near the two-grid-interval wavelength [3, sect. 13.3]. Higher orders n damp more selectively, but for $n \geq 4$ the diffusive flux no longer always points down-gradient. This means the filter can create new extrema exactly where we need it most: at sharp gradients like inversions, out-flow boundaries, or cloud edges. The resulting undershoots are unphysical for bounded fields like tracer concentrations. The reason is structural: any consistent linear discretization of ∇^n for $n \geq 4$ has sign-alternating stencil weights, so the update isn’t a convex combination of the values it reads. No choice of evaluation point or coefficient can fix that. Xue [10] therefore proposed nonlinear monotonic variants; the practical one is a simple flux limiter: set to zero any diffusive flux that points up-gradient. This scheme is already used operationally, e.g., in WRF-ARW’s sixth-order monotonic filter [5].

We investigate what monotonicity and filter order actually cost on a modern CPU. Starting from the course’s stencil2d, we built a validated, order-generic ($n = 2, 4, 6, 8$), MPI-parallel implementation. We measure the limiter’s overhead against filter order and compare with the more complex Zalesak-type flux correction. We also evaluate data-locality optimizations for the limited operator, using roofline analysis to explain why one of them fails.

2. Numerical method

a. Order-generic filter

With $m = n/2$ (the number of Laplacian pre-sweeps), one filter application advances

$$\phi^{l+1} = \phi^l + (-1)^{m+1} \alpha_n \text{div } F, \quad F = \text{grad}(\text{lap}^{m-1} \phi) : \quad (2)$$

that is, $m-1$ Laplacian pre-sweeps followed by a final Laplacian in *flux form*: differences of fluxes across cell faces rather than a weighted sum of neighbors. This is the only form that exposes the individual fluxes the limiter needs to inspect. Since the discrete $\text{div} \circ \text{grad}$ is the 5-point Laplacian, Eq. (2) equals $\phi^l + (-1)^{m+1} \alpha_n \text{lap}^m \phi^l$, a consistent $\mathcal{O}(\Delta x^2)$ discretization of Eq. (1). The $n=4$ construction and the nondimensional coefficient follow the course material [1, slides 20 and 24] and Xue [10]; higher orders are just further Laplacian applications. On a periodic grid, the damping factor (Proposition 1, Appendix A; this is Eq. (12) of Klemp [4]) is

$$\lambda(\mathbf{k}) = 1 - \alpha_n L(\mathbf{k})^m, \quad L = 4 \sin^2 \frac{k_x \Delta x}{2} + 4 \sin^2 \frac{k_y \Delta y}{2}, \quad (3)$$

with $L \leq 8$ in 2D. We normalize $\alpha_n = 1/8^m$, the maximum-damping coefficient of Klemp [4], so that the checkerboard mode is removed in one step *at every order*: a larger coefficient amplifies some modes, a smaller one needs several steps. This makes orders directly comparable. Longer waves survive increasingly well with order: for the (4,3) mode on a 64^2 grid, $\lambda = 0.970, 0.9991, 0.99997, 0.9999992$ at $n = 2, 4, 6, 8$.

b. Simple flux limiter

Following Eq. (10) of Xue [10], we keep an interface flux only if it is strictly down-gradient:

$$F \leftarrow F \cdot \max[0, \text{sign}(\text{sgn} \cdot F \delta \phi)], \quad \text{sgn} = (-1)^{m+1}, \quad (4)$$

where $\delta \phi$ is the field difference across the interface, applied independently at each of the four cell faces. The factor sgn absorbs the alternating sign of the order- n operator, so the same test works for all orders. Note that $\text{sign}(0) = 0$: fluxes across flat interfaces ($\delta \phi = 0$) are also zeroed. For $n = 2$ the flux equals the gradient, so in exact arithmetic the limiter never acts – a sanity check that our implementation passes.

c. Zalesak flux correction

The alternative scheme, the flux correction of Zalesak [12] as adapted in Eqs. (5)–(9) of Xue [10], forms two fluxes per interface: the order- n flux A^H (sharp but not monotone), and a second-order flux A^L (blunt but monotone, with $\alpha_2 = 1/8$

Course project supervised by Max Harvey; code at github.com/ofuhrer/HPC4WC under [projects/2026/project06-monotonic-high-order-diffusion](https://github.com/ofuhrer/HPC4WC/blob/main/LICENSE).

as our normalization prescribes). Their difference $A^{HL} = A^H - A^L$ is the corrective flux. The scheme advances with A^L alone and then restores as much of the correction as local extrema allow, so it limits the correction rather than the whole flux:

$$\begin{aligned} A^{HL} &= A^H - A^L, & \phi^* &= \phi - \text{div} A^L, \\ \phi^{l+1} &= \phi^* - \text{div}(CA^{HL}), \end{aligned} \quad (5)$$

with $C \in [0, 1]$ computed per interface from the local-extrema headroom of ϕ^l (5-point neighborhood, guarded ratios with $\varepsilon = 10^{-15}$). It needs one extra halo ring ($n/2 + 1$, a runtime parameter) but still only one exchange per iteration. Two forced limits validate the implementation: $C \equiv 1$ reproduces the unlimited filter up to roundoff; $C \equiv 0$ reproduces the second-order filter exactly.

d. Halo design

The m sweeps use an extend cascade: sweep s computes $m - s$ rings into the halo, so each sweep finds valid neighbors without a further exchange. An order- n filter therefore needs a halo of width $n/2$ and still exactly *one* exchange per iteration for any order. Halo width is a runtime parameter; the MPI volume grows linearly with it.

3. Implementation and validation

a. The variant ladder

All variants were derived from the course’s day-3 MPI stencil12d (single precision, 5-point Laplacians, two-stage halo exchange, 2D domain decomposition with scatter/gather to rank 0). They form the optimization ladder: filter (naive, full-size 3D temporaries for every sweep and both flux components, the ARPS pattern [11]); filter-kblock (temporaries demoted to per- k -level 2D planes); filter-inline (flux arrays further demoted to scalars in one fused flux-limit-update loop, recomputing each interior flux twice), the latter two each with a branchy (IF) and a branch-free (merge) limiter formulation; and filter-zalesak (the flux-corrected alternative, naive layout). Each rung differs from the one below by a single idea and must reproduce the reference bit for bit, so the runtime comparisons of Sec. 4 compare one computation, not two.

b. Validation

An automated suite runs on every change, on a laptop (MPICH) and on Alps (Cray MPICH), targeting the failure modes typical of stencil codes: incorrect parallelization, layout-dependent drift, and silent sign or coefficient errors. It requires bit-exactness between one and many ranks, between a minimal and a wider halo, and between every optimized variant and the naive filter in all limiter modes; it checks monotonicity in both directions, since the limited filter must preserve the input bounds and the unlimited one must violate them; and it compares measured single-mode damping against Eq. (3), agreeing to $\sim 10^{-8}$ relative at every order.

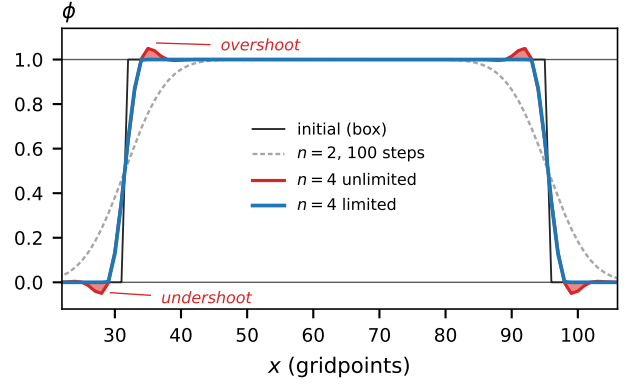


FIG. 1. Cross-section through the box after 100 steps (128^2 , $n=4$, $\alpha_4 = 1/64$). Shaded lobes mark where the unlimited filter leaves $[0, 1]$; the limited filter preserves it exactly, with a far sharper edge than second-order diffusion.

TABLE 1. Naive order-generic filter on one Grace socket: runtime per 1024 iterations ($128^2 \times 64$), limiter cost, and fraction of nonzero fluxes zeroed.

n	halo	off [ms]	limited [ms]	cost	activity
2	1	33.5	55.5	+66%	$1.4 \cdot 10^{-4}$
4	2	63.0	88.8	+41%	$9.8 \cdot 10^{-2}$
6	3	82.3	108.3	+32%	$4.5 \cdot 10^{-2}$
8	4	100.0	119.7	+20%	$5.6 \cdot 10^{-2}$

4. Results

All measurements were taken on one GH200 node of Alps (CSCS, vCluster Santis): 72 MPI ranks pinned to the cores of a single 72-core NVIDIA Grace CPU (Neoverse-V2), gfortran 14.3 (-O3 -mcpu=native, uenv prgenv-gnu/26.3:v1), single precision, box initial condition (Fig. 1), medians of 2–3 repetitions in one Slurm allocation. The reference configuration is $128^2 \times 64$ with 1024 iterations, with $\alpha_n = 1/8^{n/2}$ throughout (Sec. 2a). Every number traces to a job script and raw log under benchmarks/.

a. Cost of monotonicity

After 100 steps the unlimited fourth-order filter has left the field with extrema $[-0.052, 1.092]$; the limited one preserves $[0, 1]$ exactly, with a far sharper edge than second-order diffusion (Fig. 1). Monotonicity costs twice. First, flux form – the prerequisite for inspecting individual fluxes – costs +5.4% at $n=4$ in the naive layout, even with the limiter off. The limiter itself adds another +20 to +26 ms per 1024 iterations (Table 1).

In absolute terms, the limiter’s cost is almost independent of order – it’s the same flux/limit/update loop every time – while the unlimited filter runtime grows roughly linearly with the number of sweeps $n/2$, as expected for a memory-bound kernel. Thus the relative overhead drops from +66% at $n=2$ to +20% at $n=8$: monotonicity is cheapest exactly when the filter is most selective.

The firing frequency of the limiter depends on the flow and does not drive the cost. Our box initial condition is a piecewise-

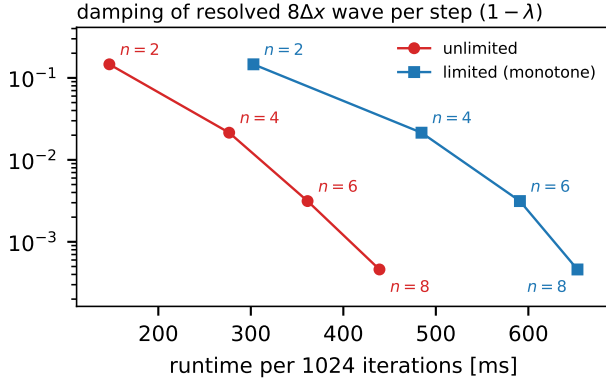


FIG. 2. Selectivity-cost trade-off: spurious damping of a resolved $8\Delta x$ mode per step (log scale) vs. measured runtime ($512^2 \times 64$; $n \neq 4$ runtimes scaled from the order scan). Every order removes $2\Delta x$ noise in one step.

constant worst case, zeroing 9.8% of nonzero fluxes at $n=4$ and $\sim 5\%$ at $n=6, 8$, compared to below 1% for the smooth convection test of Xue [10].

b. Benefit of higher order

Because every order removes the two-grid-interval mode in one step, the orders differ only in collateral damage to resolved scales and runtime. This makes Fig. 2 a fair comparison. From $n=2$ to $n=8$ the spurious damping of a well-resolved $8\Delta x$ wave falls by a factor of about 300, while runtime only triples (with monotonicity adding another 20–66%).

High order clearly dominates the trade-off. Its real costs are elsewhere: the halo deepens to $n/2$, increasing MPI volume and memory footprint with order – but those are not arithmetic costs.

c. Data-locality optimizations

Array demotion becomes the dominant transformation once the working set no longer fits in cache. In the constant-work domain scan of Table 2 (size² × iters fixed, normalized to ms per 1024 iterations), at 1024^2 k-blocking is 2.7–2.8× faster than the naive 3D-temporary layout, and full inlining reaches 3.1–3.3×; the last 10–17% comes from recomputing each interior flux twice being cheaper than streaming two flux planes.

The speedup is entirely conditional on problem size (Fig. 3). At 128^2 , with 72 ranks, each rank gets a cache-resident subdomain, and all three layouts fall within $\sim 10\%$ of one another. The peak 3.3× appears only when the working set spills to DRAM. Quoting that peak alone would overstate what the optimization is worth at production-like decompositions.

d. Why the limiter resists fusion

Fusing the loops interacts badly with the limiter. With the branchy limiter the fully inlined variant is actually *slower* than the less aggressive k-blocked one: 3935 vs. 3175 ms at 1024^2 and $n=4$. The reason is that the four data-dependent IFs in the fused loop prevent the compiler from vectorizing the update; the branch outcome varies from point to point, forcing Grace back to one grid point per SVE instruction. Rewriting the same

TABLE 2. Optimization ladder at $n=4$ (ms per 1024 iterations, $n_z=64$, constant work, median of 2). “off” = limiter disabled; branchy = IF-based, branch-free = merge-based. $n=8$ shows the same pattern (peak $3.1\times$ unlimited, $2.6\times$ limited).

size	variant	off	branchy	branch-free
128^2	naive	62.4	86.6	n/a
	k-block	58.8	88.3	71.5
	inline	55.5	102.6	72.0
512^2	naive	556	826	n/a
	k-block	334	719	485
	inline	277	977	507
1024^2	naive	4645	5172	n/a
	k-block	1660	3175	2191
	inline	1401	3935	2096

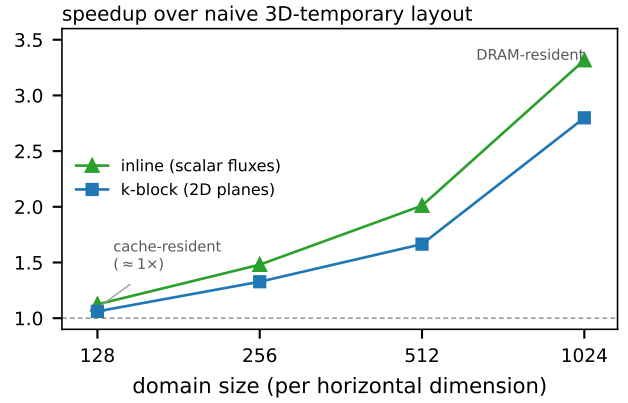


FIG. 3. Array-demotion speedup over the naive 3D-temporary layout vs. problem size ($n=4$ unlimited, 72 ranks, constant work). At 128^2 the per-rank subdomains are cache-resident and all variants are within $\sim 10\%$ of naive; the gain reaches $2.8\times$ (k-block) and $3.3\times$ (inline) once the working set spills to DRAM at 1024^2 .

test with *merge*, which selects without branching, recovers 31% (k-block) and 47% (inline), making inline plus branch-free the fastest limited configuration – $2.5\times$ ($n=4$) to $2.6\times$ ($n=8$) faster than the naive limited filter.

Hardware counters confirm this (Fig. 4). We estimate traffic from the ARM performance-monitoring unit as last-level-cache misses $\times 64$ B (a proxy that overcounts due to prefetch). The naive filter moves 641–723 GB at a sustained 570–590 GB/s and sits on the memory roof, with arithmetic intensity ≈ 0.35 flop/B. Demotion cuts traffic 7–8× (87 GB for k-block, 78 GB for inline) and the kernel leaves the memory-bound regime. The decisive comparison is between the two limiter formulations: at *identical* traffic (≈ 62 –83 GB) the branchy loop sustains 66–108 GB/s against the branch-free 125–162 GB/s. Equal bytes and twice the time can only mean a core-bound kernel.

Two caveats: the naive and Zalesak points sit slightly above the roof, which no kernel can exceed – the miss proxy overcounts true traffic and the roof is nominal rather than measured,

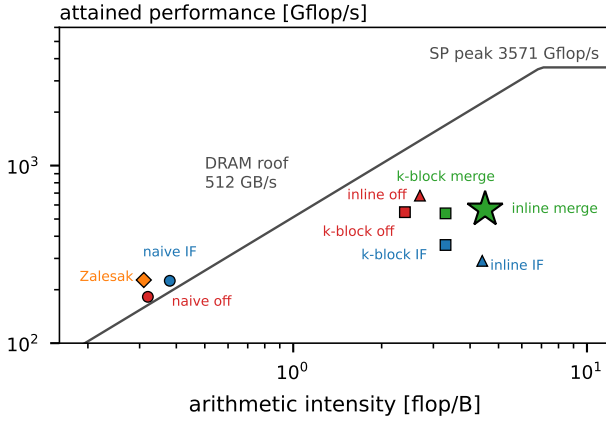


FIG. 4. Roofline [9] placement of the $n=4$ variants (1024^2 , 256 iterations, one Grace socket); labels follow Table 2. The demoted unlimited variants approach the roofs, the branchy-limiter points fall far below both (core-bound), and Zalesak sits at the DRAM roof. The star marks the best limited configuration; points above the roof are a proxy artifact (see text).

both effects growing with the number of streamed arrays. Also, optimizing the operator moves the bottleneck: on the naive variant the limiter looks cheap at large sizes (+11%) because temporary-array traffic dominates, while after demotion it becomes the main remaining cost (+91% branchy, +32% branch-free).

e. The Zalesak alternative

We implemented the flux correction to see if its extra complexity pays off. It doesn't. After 100 steps from the box the two limited solutions are nearly indistinguishable, with a maximum pointwise difference of $2.3 \cdot 10^{-2}$ and identical $\phi=0.5$ half-widths along axis and diagonal. The correction costs $1.87\times$ the simple limiter at 128^2 and $2.22\times$ at 1024^2 . Its guarantee is also weaker: for $n=4,6$ it is monotone only to single-precision roundoff (minimum $-1.2 \cdot 10^{-8}$), since it compensates fluxes through ϕ^* arithmetic, whereas zeroing a flux bounds the simple limiter exactly. The mean correction coefficient is flow-dependent (0.16 after 1024 steps at 128^2 , 0.003 after 16 steps at 1024^2): the correction is suppressed when gradients are sharp – exactly when it would be needed.

Xue [10] asserts this ranking but never times it, calling the simple limiter “far more efficient” and the correction “much more expensive” on solution quality alone, and in two dimensions finds the simple limiter *better* (his Fig. 6). Our measurements supply the missing factor, though within our metrics the two solutions are equivalent rather than ordered. The ranking survives 26 years of hardware change; its reasoning does not. The paper counts operations, whereas here the correction is expensive because it streams nine full-size 3D arrays per iteration, moving $2.9\times$ the limited filter's DRAM traffic for $2.3\times$ its runtime (Fig. 4).

5. Generality vs. readability

Can the optimizations of Sec. 4 be applied generally without sacrificing readability? We implemented the same order-generic limited filter a second time in GT4Py [2, `gt4py.next.v1.1.10`, `gtfn` backends, `float32`] and compared the two on correctness, performance and generality.

a. Correctness

The GT4Py version is *bit-exact* against the Fortran reference for every order and limiter mode; the generated C++ reproduces gfortran's single-precision arithmetic on identical expression trees. Any reassociation or fused multiply-add contraction would break this, so achieving bit-exactness is a strong validation result: the two implementations evaluate the same expression, not merely expressions agreeing to a tolerance.

b. Performance

Against our best Fortran variant on one Grace core at $256^2 \times 64$ (Fig. 5), the `gtfn` CPU backend is $2.6\text{--}5.7\times$ slower; on one H100 the same source runs $32\text{--}76\times$ faster than that core – a statement about portability rather than about the two sockets. More striking is that the limiter costs nothing there (14.4 vs. 14.2 ms, within run-to-run variation): predication is native on GPUs, so the penalty of Sec. 4d does not arise.

c. Do the optimizations generalize?

In Fortran, k-blocking, inlining and demotion are decisions written into the loop nest, so each combination is a separate source file. In GT4Py they are absent from the source. The four interface fluxes of Eq. (4) are local subexpressions inside one field operator rather than arrays, and whether they become temporaries or registers is the backend's choice; switching backend re-schedules the whole kernel with no source change. In that sense the optimizations do generalize, and one Fortran design axis disappears with them: *where* is the only way to express the limiter and lowers to predication on either backend, so the branchy formulation that costs 31–47% (Sec. 4d) cannot be written by accident.

What does not generalize is demotion *between* sweeps. Field operators admit no runtime loops, so order-genericity is expressed by *regeneration*: a 60-line generator emits the (order $\times$ limiter) program matrix from one source. But nested field-operator calls do not lower in `gt4py` v1.1.10, so the pre-sweeps must be separate program statements over explicit domains, each materializing a full-size 3D temporary. That is exactly the naive layout of Sec. 4c, and the schedule we would need to change is not ours to write. The same layout costs $2.7\text{--}3.3\times$ in Fortran, which is the order of the CPU shortfall above, though we did not isolate it.

Generality is therefore real here, and what it costs is control rather than readability: the stencil definition is ~ 40 schedule-free lines against 600–750 for a complete Fortran variant, but where it is slow there is no source-level remedy.

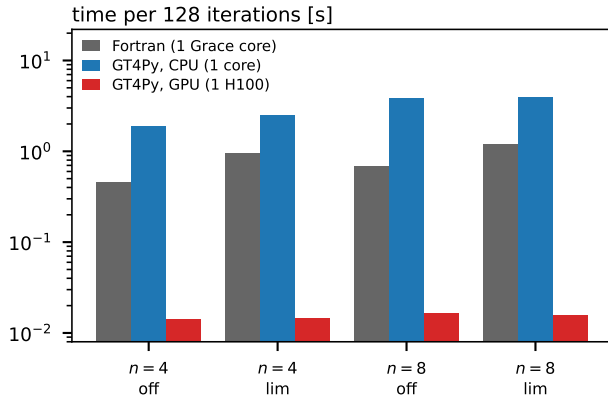


FIG. 5. GT4Py against hand-optimized Fortran: time per 128 iterations ($256^2 \times 64$; one Grace core for the CPU bars, one H100 for the GPU bars; log scale). On the GPU the limited and unlimited bars are equal (predication is native); on the CPU the limiter adds cost, as in Fortran.

6. Conclusions

Monotonicity via the simple flux limiter is exact and becomes proportionally cheaper as the filter sharpens: +66% at $n=2$ vs. +20% at $n=8$. Higher order reduces the collateral damping of resolved scales by a factor of about 300 for only a $3\times$ runtime increase. So monotone high-order filtering is cheapest exactly where it is most useful. The Zalesak correction gives indistinguishable solutions at twice the cost and a weaker finite-precision guarantee.

The optimization picture is layered. Array demotion gives the biggest gain ($3.3\times$), but only once the working set leaves cache. Inlining helps only until the limiter's branches defeat vectorization; the branch-free merge form, worth 31–47%, reconciles the two. Applying these optimizations generally means handing them to a compiler: in GT4Py they leave the source altogether, which buys portability and makes the slow limiter formulation impossible to write, but the one transformation the backend did not perform – demotion between sweeps – is also the one we can no longer perform ourselves. Enabling monotone high-order diffusion in models like WRF-ARW [5] or ICON [13] thus carries a modest, predictable penalty that shrinks with filter order: precisely the regime where the scale selectivity that motivates it is greatest.

Declaration on the use of generative AI

We used generative AI tools for language polishing and sentence restructuring during the writing of this report. The scientific work is our own. We have reviewed all AI-generated content and accept full responsibility for the final submission.

References

[1] Oliver Fuhrer and coauthors. High performance computing for weather and climate: lecture material, introduction and stencils. ETH Zurich course 701-1270-00L, github.com/ofuhrer/HPC4WC, 2026.

[2] GridTools project. GT4Py: GridTools for Python. github.com/GridTools/gt4py, 2026. Version 1.1.10, gtfm backends.

[3] Christiane Jablonowski and David L. Williamson. The pros and cons of diffusion, filters and fixers in atmospheric general circulation models. In Peter Lauritzen, Christiane Jablonowski, Mark Taylor, and Ramachandran Nair, editors, *Numerical Techniques for Global Atmospheric Models*, volume 80 of *Lecture Notes in Computational Science and Engineering*, pages 381–493. Springer, Berlin, Heidelberg, 2011. doi: 10.1007/978-3-642-11640-7_13.

[4] Joseph B. Klemp. Damping characteristics of horizontal Laplacian diffusion filters. *Mon. Wea. Rev.*, 145(11): 4365–4379, 2017. doi: 10.1175/MWR-D-17-0015.1.

[5] Jason C. Knierel, George H. Bryan, and Joshua P. Hacker. Explicit numerical diffusion in the WRF model. *Mon. Wea. Rev.*, 135(11):3808–3824, 2007. doi: 10.1175/2007MWR2100.1.

[6] Ralph Shapiro. Smoothing, filtering, and boundary effects. *Rev. Geophys.*, 8(2):359–387, 1970. doi: 10.1029/RG008i002p00359.

[7] Ralph Shapiro. Linear filtering. *Math. Comp.*, 29(132):1094–1097, 1975. doi: 10.1090/S0025-5718-1975-0389356-X.

[8] Frederick G. Shuman. Numerical methods in weather prediction: II. Smoothing and filtering. *Mon. Wea. Rev.*, 85(11):357–361, 1957. doi: 10.1175/1520-0493(1957)085<0357:NMIWPI>2.0.CO;2.

[9] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[10] Ming Xue. High-order monotonic numerical diffusion and smoothing. *Mon. Wea. Rev.*, 128(8):2853–2864, 2000. doi: 10.1175/1520-0493(2000)128<2853:HOMNDA>2.0.CO;2.

[11] Ming Xue, Kelvin K. Droegemeier, Vince Wong, Alan Shapiro, and Keith Brewster. *ARPS version 4.0 user's guide*. Center for Analysis and Prediction of Storms, University of Oklahoma, Norman, Oklahoma, 1995. 380 pp. Code inspected: ARPS v5.3.4, src/arp/cmix3d.f90.

[12] Steven T. Zalesak. Fully multidimensional flux-corrected transport algorithms for fluids. *J. Comput. Phys.*, 31(3): 335–362, 1979. doi: 10.1016/0021-9991(79)90051-2.

[13] Günther Zängl, Daniel Reinert, Pilar Rípodas, and Michael Baldauf. The ICON (ICOsahedral Non-hydrostatic) modelling framework of DWD and MPI-M: description of the non-hydrostatic dynamical core. *Quart. J. Roy. Meteor. Soc.*, 141(687):563–579, 2015. doi: 10.1002/qj.2378.

A. Derivation of the damping factor

Proposition 1. *On the periodic grid, every Fourier mode $\phi_{jl} = \hat{\phi} e^{i(k_x j \Delta x + k_y l \Delta y)}$ is an eigenvector of the unlimited order- n update (2), with eigenvalue $\lambda(\mathbf{k}) = 1 - \alpha_n L(\mathbf{k})^m$ and L as in Eq. (3).*

Proof. Each one-dimensional second difference acts on the mode as multiplication by

$$e^{-ik_x \Delta x} - 2 + e^{ik_x \Delta x} = 2 \cos(k_x \Delta x) - 2 = -4 \sin^2 \frac{k_x \Delta x}{2}. \quad (6)$$

The 5-point Laplacian is the sum of the two one-dimensional operators, so its symbol is $-L(\mathbf{k})$ with $L \in [0, 8]$, the maximum attained by the checkerboard mode. The update (2) is algebraically $\phi^{l+1} = \phi^l + (-1)^{m+1} \alpha_n \text{lap}^m \phi$, and lap^m multiplies the mode by $(-L)^m = (-1)^m L^m$; hence

$$\lambda = 1 + (-1)^{m+1} (-1)^m \alpha_n L^m = 1 - \alpha_n L^m. \quad (7)$$

□

Corollary 1. $\lambda = 0$ at the checkerboard mode iff $\alpha_n = 1/8^m$ (the normalization of Sec. 2), and $|\lambda| \leq 1$ for all modes iff $\alpha_n \leq 2/8^m$.

Remark. Proposition 1 is not new: Eq. (3) is Eq. (12) of Klemp [4] for the recursively applied five-point Laplacian, and the normalization $\alpha_n = 1/8^m$ of Corollary 1 is his maximum-damping coefficient $2^{-3n/2}$. In one dimension it reduces to the classical filter response $1 - \sin^2(k \Delta x/2)$ [6–8]; explicit hyperdiffusion and its stability limits are reviewed by Jablonowski and Williamson [3]. The result holds only for the linear scheme, since Fourier modes are not eigenvectors of the limited update (4), which is why the damping checks of Sec. 3 run with the limiter off.

B. Reproduction details

Unless stated otherwise, all measurements use the platform of Sec. 4 with Cray MPICH. Build with `make allversions` in `src/`; `tools/validate.sh` (14 analytic checks) must pass before any measurement. Each experiment has its own directory under `benchmarks/`, holding its job script and the parsed result notes. All figures regenerate with `python3 tools/make_figures.py` (output directory `figures/`, or `FIGDIR`), whose constants are the medians quoted below.

a. Figure reproduction

- **Fig. 1 (overshoot cross-section):** three 1-rank runs of `stencil2d-filter.x` at $128^2 \times 4$, 100 iterations, box IC, default α_n ($n=4$ unlimited, $n=4$ limited, $n=2$); the plot is row $j=64$ of the middle k -level.
- **Fig. 3 (demotion crossover):** naive-over-variant runtime ratios (unlimited, $n=4$) from the Table 2 domain scan, at sizes 128/256/512/1024.
- **Fig. 4 (roofline):** DRAM traffic from LLC-load-miss counters ($\times 64$ B), arithmetic intensity from analytic flop counts (about 12, 16 and 38 flop per point per iteration for the unlimited, limited and Zalesak kernels). The roofs are the nominal socket peaks, 512 GB/s and 3571.2 SP Gflop/s.
- **Fig. 2 (selectivity-cost):** vertical axis is $1 - \lambda$ for the $(\pi/4, \pi/4)$ mode from Eq. (3) with $\alpha_n = 1/8^{n/2}$; horizontal axis is the 512² best-variant runtimes, with the $n \neq 4$ points scaled by the order-scan ratios (noted in the caption).
- **Fig. 5 (GT4Py backends):** time per 128 iterations at $256^2 \times 64$, box IC, for the best Fortran variant on one Grace core, the gtfm CPU backend on one core, and the gtfm GPU backend on one H100.

b. Table reproduction

- **Table 1 (cost vs. order):** naive filter at $128^2 \times 64$, 1024 iterations, box IC, medians of 3 per order. Activity is the fraction of nonzero interface fluxes the limiter zeros.
- **Table 2 (optimization ladder):** constant-work domain scan ($\text{size}^2 \times \text{iters}$ held fixed), $n=4$, normalized to ms per 1024 iterations, medians of 2; “off” disables the limiter, branchy is the IF form, branch-free the merge form.

c. In-text numbers

- **Zalesak comparison:** `filter-zalesak` against `filter --limiter` at $n=4$; cost is the runtime ratio, the mean C is the per-interface average over the run, and quality is the $\phi=0.5$ half-widths and the $\max/L_2$ solution differences after 100 steps (all printed by the figure script). The counter run adds the $2.9 \times$ traffic figure.
- **Flux-form overhead (+5.4%):** flux-form filter against the lap-of-lap baseline at $n=4$, both unlimited, on one Grace socket.